

Centro Universitário UniFECAF

Cloud DevOps: Orchestrating Containers and Micro Services

KEVIN MAIKON CAETANO DE ANDRADE SANTOS

**Entrega contínua de uma plataforma de pedidos em microsserviços: do
Docker Compose ao Kubernetes com observabilidade e CI/CD**

Evidências de execução do MVP

Taboão da Serra
2026

1 OBJETIVO DESTE DOCUMENTO

Este documento reúne evidências da execução do MVP Pedidos Veloz, complementando o relatório técnico prático e o repositório. São apresentados o diagrama de arquitetura, o estado dos containers Docker Compose, as respostas da API e o resultado dos testes automatizados.

2 ARQUITETURA DA SOLUÇÃO

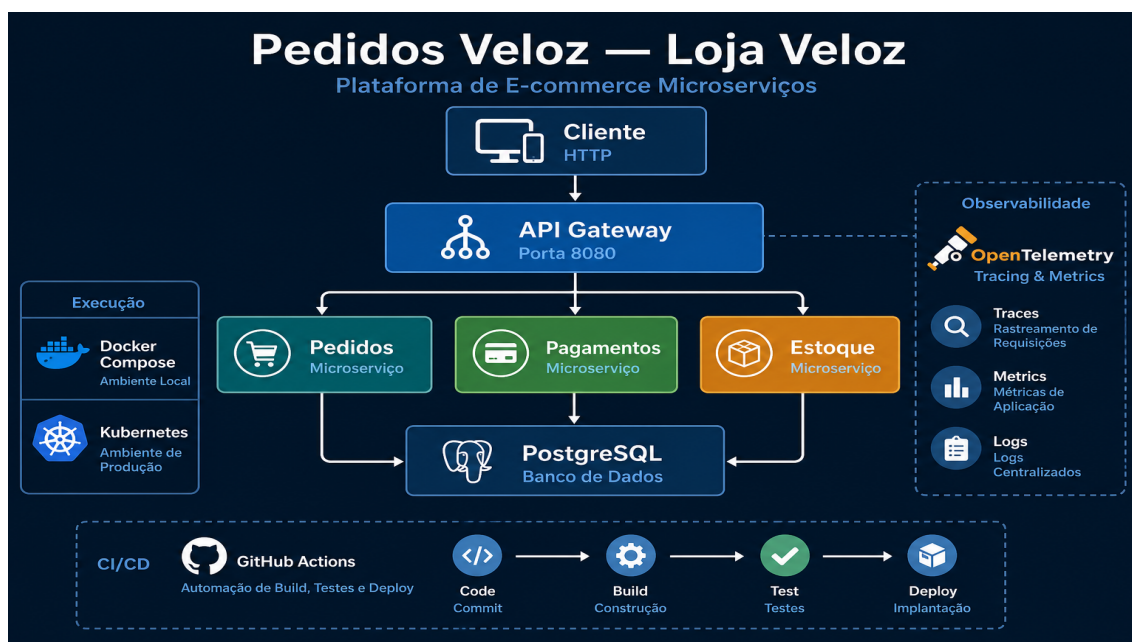


Figura 1 – Arquitetura Pedidos Veloz (Gateway, Pedidos, Pagamentos, Estoque e PostgreSQL)

A arquitetura segue o padrão de microsserviços com entrada única pelo API Gateway na porta 8080, orquestração do fluxo de pedido e persistência em PostgreSQL.

3 AMBIENTE LOCAL (DOCKER COMPOSE)

Após *docker compose up --build*, os cinco containers permaneceram em estado *healthy*: pv-gateway, pv-pedidos, pv-pagamentos, pv-estoque e pv-postgres. O gateway publicou a porta 8080 e o PostgreSQL a porta 5432.

NOME	STATUS	PORTAS
pv-estoque	Up (healthy)	3003/tcp
pv-gateway	Up (healthy)	0.0.0.0:8080->8080/tcp
pv-pagamentos	Up (healthy)	3002/tcp
pv-pedidos	Up (healthy)	3001/tcp
pv-postgres	Up (healthy)	0.0.0.0:5432->5432/tcp

4 VALIDAÇÃO DA API

Resposta de GET /health:

```
{
  "status": "ok",
  "service": "api-gateway",
  "upstreams": {
    "pedidos": "http://pedidos:3001",
```

```

    "estoque": "http://estoque:3003",
    "pagamentos": "http://pagamentos:3002"
  }
}

```

Resposta de POST /api/pedidos (pedido confirmado):

```

{
  "id": "973515d1-e56e-4c4e-b4b8-8ad98f335045",
  "status": "CONFIRMADO",
  "valorTotal": 49.9,
  "pagamentoId": "e012413f-a3c7-4e3f-a235-e294036a0a1e",
  "traceId": "9d51c27b59cb184d8ebec7282801bf26"
}

```

5 TESTES AUTOMATIZADOS

```

...
# Subtest: recusa valor com centavos .13
ok 2 - recusa valor com centavos .13
---
  duration_ms: 0.221
  type: 'test'
...
1..2
# tests 2
# suites 0
# pass 2
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 142.6567
> estoque-service@1.0.0 test
> node --test test/*.test.js
TAP version 13
# Subtest: preço de SKU conhecido
ok 1 - preço de SKU conhecido
---
  duration_ms: 0.9969
  type: 'test'
...
# Subtest: validade de quantidade
ok 2 - validade de quantidade
---
  duration_ms: 0.1893
  type: 'test'
...
1..2
# tests 2
# suites 0
# pass 2
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 140.6398

```

Os testes unitários dos quatro serviços (api-gateway, pedidos, pagamentos e estoque) foram executados com sucesso via *npm test*.

6 CONSIDERAÇÕES

As evidências acima comprovam a execução local multi-serviço, a integração entre os microsserviços e a existência de testes automatizados, em conformidade com os

requisitos práticos da disciplina.